

Step 3 — Research Agent — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic**, **Intermediate**, **Advanced**.

- [Hint 1 — The tool, and the shape of the agent](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The tool, and the shape of the agent

Basic Version

The Research agent is different from Writer and Reviewer: it actually needs to *look something up*, not just transform text it's already given. That means it needs a tool — something real (a search API) or something fake but tool-shaped (a function that returns made-up facts for a given topic, so you don't need a real API key yet to prove the agent's structure works).

Follow the same order you used for Writer: get the underlying lookup working first, then wrap it as an agent.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The README asks for `run_research(topic) -> ResearchNotes`. Two parts to design: the tool itself (Doc06's `@tool` decorator pattern), and the agent loop that decides to call it and then produces the final notes.

A fake lookup tool is genuinely fine here — the point of Step 3 isn't proving a real search API works, it's proving the Research agent's *shape* (tool-calling agent → structured notes output) is right, the same way Step 1 used made-up notes to prove the Writer's shape without needing Research to exist yet. Swapping a fake tool for a real search API later is a small, contained change specifically because the interface (`run_research(topic) -> ResearchNotes`) doesn't change.

`ResearchNotes` should be a structured object (Pydantic model), not a raw string — this is what lets Step 4's Analysis agent depend on a stable shape rather than parsing free text. Write the tool's docstring like you're explaining *when* to use it to the model, not documenting it for a human — that docstring is what the model actually reads to decide whether to call the tool at all.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's the uncomfortable truth about the docstring instruction "always call this tool, don't rely on your own general knowledge": it's a *request*, not a guarantee. A capable model will sometimes skip a tool it doesn't think it needs and just answer from what it already knows — especially on topics it has confident-sounding training knowledge about. Since the entire point of a Research agent is that its output comes from the tool, not the model's own guesses, a docstring alone can't prove that happened. You need to check.

The fix is a runtime verification, not a stronger-worded prompt: after the agent runs, look at its message history for a tool-call message with the lookup tool's name. If one isn't there, the agent skipped the tool — and `run_research` should treat that as a failure (raise, or retry once with a more forceful instruction), not silently return whatever the model guessed as if it were real research.

The extra pieces:

- After `agent.invoke(...)`, loop over `result["messages"]` and check whether any message's tool name matches your lookup tool — this is the same technique Step 3's Intermediate solution already uses to *extract* facts, just repurposed here to *verify* the tool ran at all, before trusting the result.
- A custom exception, like `ToolNotCalledError`, raised if the check fails — naming the specific problem, the same "fail loudly and specifically" idea as the `.env` parsing exercise's `MissingEnvKeyError`.
- Also cap the agent's `recursion_limit` (a `create_agent` / graph config option) — a tool-calling agent that keeps calling a tool in a loop without converging is a real failure mode worth bounding, the same "don't loop forever" idea Step 2's Writer ↔ Reviewer loop already had to solve.

Sketch the tool-call verification yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both trust the docstring's instruction to actually work. Advanced adds a check that confirms it did — the same shift Step 1's Advanced made from "hope the input is valid" to "verify the input is valid," just applied to trusting a tool call instead of trusting a caller's arguments.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make a fake lookup tool:
    given a topic, return a short list of made-up facts about it

make a ResearchNotes shape: topic, facts (a list of text)

make run_research(topic):
    build an agent that has the fake tool available
    ask the agent to research the topic using the tool
    turn its answer into a ResearchNotes and return it

test:
    call run_research on 2-3 made-up topics
    check the facts list actually has something in it, and the tool really got ca
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
agents/research_agent.py:
@tool
def lookup_facts(topic: str) -> list[str]:
    """Look up basic facts about a topic. Always call this before writing
    research notes – do not rely on your own general knowledge."""
    return a fake but plausible list of fact strings for the topic

class ResearchNotes(BaseModel):
    topic: str
    facts: list[str]

def run_research(topic: str) -> ResearchNotes:
    agent = create_agent(ChatOpenAI(model="gpt-4o-mini"), tools=[lookup_facts])
    result = agent.invoke({"messages": [("user", f"Research this topic: {topic}")]})
    facts = extract_the_tool's_returned_facts_from(result["messages"])
    return ResearchNotes(topic=topic, facts=facts)
```

Test that `lookup_facts` genuinely gets *called* (not just that a plausible-looking answer comes back) — print the agent's intermediate messages/tool calls at least once while testing, so you know the tool-calling mechanism is really working and not just the model guessing plausible-sounding facts on its own. Write it yourself, then compare against the [Solution](#).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's almost the verified version — fill in the missing check yourself:

```
class ToolNotCalledError(Exception):
    pass

def run_research(topic: str) -> ResearchNotes:
    agent = create_agent(
        ChatOpenAI(model="gpt-4o-mini"),
        tools=[lookup_facts],
    )
    result = agent.invoke(
        {"messages": [("user", f"Research this topic using the lookup_facts tool: {topic}"),
                      {"assistant", f"Research this topic using the lookup_facts tool: {topic}"}],
         {"recursion_limit": 8},
    )

    # your turn: check whether any message in result["messages"] is the
    # lookup_facts tool's own returned message (its .name attribute equals
    # "lookup_facts"). If none is found, raise ToolNotCalledError(topic) –
    # the model answered from its own knowledge instead of using the tool.
    ...

    facts = [...] # extracted from the verified tool message(s)
    return ResearchNotes(topic=topic, facts=facts)
```

Fill in the verification check, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same agent-plus-tool mechanics at 3 completeness levels — Basic and Intermediate build the agent and trust its output; Advanced adds a check *after* the agent runs that confirms the tool was actually the source of the facts, raising a specific, named error if it wasn't, instead of quietly returning `ResearchNotes` built from an unverified guess.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)